P0 — Why this project is not pushed by the assistant, and how it reaches GitHub

Provenance required by Hard Rule 7 for a deviation from Hard Rule 2 (PROJECT spacetimes)

Prepared for Patrick Lee Nash, Ph.D.

November 12, 2026

1. What this document is for

Hard Rule 2 of the task specification says: *push immediately to the repository as soon as any object is updated*. Hard Rule 7 says that an instruction which cannot be followed exactly must be explained in a provenance. This document is that explanation. It records (i) what was tried, with the exact error messages; (ii) why the pushes fail; (iii) what the author decided; and (iv) the procedure by which every commit nevertheless reaches `https://github.com/once-ere/spacetimes` within about a minute of being made.

Nothing in this document concerns physics. It is the only document of the project in which an instruction of the task specification is not carried out literally by the assistant; the reason is a missing permission, not a shortcut.

2. The three places where git can run

Three different computers (or sandboxes) take part in this project. The table fixes their names, because the rest of this document refers to them.

Name	What it is	What it can do
W, the *workspace*	A Linux machine (x86_64) in the cloud in which the assistant compiles and runs all code and builds all documents. The repository clone is `/home/claude/spacetimes`.	Clone and fetch from GitHub. **Cannot push** (Sec. 3).
S, the *sandbox*	A Linux virtual machine (aarch64) on the author's Mac. It sees the project folder `/Users/nsh/Developer/github/spacetimes` and nothing else of the Mac.	Run git inside the project folder. **Cannot push**: it holds no GitHub credentials (Sec. 3).
M, the *Mac*	The author's own macOS session (Terminal).	**Can push**: it holds the author's GitHub credentials.

3. What was tried, and what happened

All times are Pacific Standard Time on November 12, 2026, to within about one minute.

Time	Where	Command	Result (verbatim)
07:27	S	`git push --dry-run origin main`	`fatal: could not read Username for 'https://github.com': terminal prompts disabled`

Time	Where	Command	Result (verbatim)
07:27	W	attach repository with push access	`pushes from this session will be refused: Claude doesn't have GitHub access to once-ere/spacetimes for your organization`
07:35	W	`git push --dry-run origin HEAD:main`	`remote: Claude doesn't have GitHub access to once-ere/spacetimes for your organization. … fatal: unable to access 'https://github.com/once-ere/spacetimes/': The requested URL returned error: 403`
07:41	W	`git push --dry-run origin HEAD:main` (after the author tried to connect GitHub)	the same 403 refusal
07:41	W	`gh auth status`	`The token in GH_TOKEN is invalid.`
07:41	S	`git push --dry-run origin main`	`fatal: could not read Username for 'https://github.com': terminal prompts disabled`

Why W fails. Pushes from the workspace go through the GitHub App of the assistant's vendor. That App has not been granted access to `once-ere/spacetimes`, and only the owner of the GitHub account can grant it. Reading the repository works because the repository is public.

Why S fails. The sandbox is deliberately isolated from the rest of the Mac. It has no access to the Mac's keychain, so git inside it has no user name and no token.

Why M works. The Mac's own git has working credentials for `github.com/once-ere`: the remote-tracking log of another repository of the author on the same machine records successful pushes on November 9 and 10, 2026.

4. The author's decisions

Time	Question	Decision
07:36	Pushes are refused; pause, or commit now and push later?	"Commit now, push later."
07:41	—	"I cannot get GitHub connected; try push." (Tried again; refused again; Sec. 3.)
07:44	—	"I will push from terminal; give me the exact syntax."

5. The procedure that replaces the direct push

Every update is committed at once in W. The commit is then carried to the author's folder, where it becomes part of the local repository, and M pushes it.

Step 1 (W): commit. Each update is committed in `/home/claude/spacetimes` as soon as it is complete.

Step 2 (W): bundle. The new commits are written into one file, a *git bundle*, which is a complete and self-checking package of commits:

```
git bundle create spacetimes_update.bundle <last-synchronized-commit>..main
```

Step 3 (W to the Mac): copy. The bundle is copied to `/Users/nsh/Developer/github/spacetimes/.git-transfer/spacetimes_update.bundle`. The folder `.git-transfer/` is excluded from version control by the local file `.git/info/exclude`.

Step 4 (S): fast-forward. Inside the project folder the sandbox verifies the bundle and advances the local branch to it. A *fast-forward* only adds commits; it never rewrites or discards anything:

```
git bundle verify .git-transfer/spacetimes_update.bundle
git fetch .git-transfer/spacetimes_update.bundle main
git merge --ff-only FETCH_HEAD
```

After this step the files in the author's folder are exactly the files of the commit, and `git rev-parse main` gives the same 40-digit commit name in W and in the folder. This equality is checked after every synchronization.

Step 5 (M): push. The author pushes from Terminal. Either once,

```
cd "/Users/nsh/Developer/github/spacetimes"
git push origin main
```

or continuously, with a loop that tries a push every 60 seconds and is silent unless there is an error (stop it with Control-C):

```
cd "/Users/nsh/Developer/github/spacetimes"
while true; do git push -q origin main; sleep 60; done
```

With the loop running, every commit reaches GitHub within about one minute of Step 4, which is as close to "push immediately" as the missing permission allows.

6. How to check that nothing is waiting

```
cd "/Users/nsh/Developer/github/spacetimes"
git status -sb
```

The first line of the answer is `## main...origin/main` when everything has been pushed, and `## main...origin/main [ahead N]` when `N` commits are still waiting.

7. What is deliberately not in the repository

The files that were already in the folder when the project started are not committed:

- the author's Mathematica notebooks (`4+4.nb`, `7+1.nb` and the others);
- the nine PDF files of published journal articles, which are not redistributed here because the right to republish them rests with their publishers;
- the author's task specification and working notes (`prompt*.txt`, `notes_for_prompt*.txt`), which the author's `.gitignore` declares private.

They remain in the folder as untracked files. Wherever the project depends on one of them, the provenance documents identify it by its SHA-256 checksum, so that the exact input can be recognized without being published. If the author wants the notebooks in the repository, one command adds them: `git add 4+4.nb 7+1.nb`, followed by a commit.

8. If the permission is granted later

If the GitHub App is given access to `once-ere/spacetimes`, the workspace W can push directly. Steps 2 to 5 then become unnecessary; the local folder is brought up to date with `git pull --ff-only`, and this document will be amended to say so.